



ADS01: AVL Trees、Splay Trees 与 Amortized Analysis

来源: /home/vincent/Code/ads/Materials/Student Slides/ADS01AVL_Stu.ppt
处理副本: /tmp/ADS01AVL_Stu.pptx (由原始 .ppt 转换, 仅用于提取)
课件范围: S1-S10 (AVL Trees); S11-S17 (Splay Trees);
S18-S25 (Amortized Analysis); S26 (参考文献)
笔记按原课件讲授顺序整理。

本讲路线图

- 普通 BST 的高度退化 → AVL 的平衡因子与旋转 → AVL 高度的对数界
- Splay 的自调整目标 → 简单旋到根的失败原因 → Zig / Zig-zag / Zig-zig 与删除
- 摊还分析框架 → 聚合分析、记账法、势能法 → 用势能证明 Splay 的 $O(\log N)$ 摊还时间

1. AVL Trees: 控制 BST 的高度 [S1-S4]

重要性: A · 必须掌握

BST 的目标是加速带插入和删除的搜索; 沿从根到目标节点的路径操作时, $T_p = O(\text{height})$ 。普通 BST 的高度却可能退化为 $O(N)$ [S2]。按 Jan 到 Dec 插入月份可得偏斜树, 平均搜索时间为 6.5; 更平衡的两棵树分别为 3.5 和 3.1。这说明相同键集合的搜索效率取决于树形 [S3]。

AVL 树 (Adelson-Velskii-Landis Tree, AVL Tree) 用局部高度平衡防止这种退化 [S4]。空树高度定义为 -1 。若非空二叉树 T 的左右子树为 T_L, T_R , 高度为 h_L, h_R , 则 T 高度平衡当且仅当:

1. T_L, T_R 都高度平衡;
2. $|h_L - h_R| \leq 1$ 。

节点的平衡因子 (Balance Factor, BF) 定义为

$$BF(v) = h_L - h_R,$$

因此在 AVL 树中每个节点的 $BF(v) \in \{-1, 0, 1\}$ [S4]。注意本课件采用 $h_L - h_R$ 的符号约定；空树高度为 -1 也是本讲约定。

来源：[S1-S4]

2. AVL 插入与旋转 [S5-S9]

重要性：A · 必须掌握

核心原理

从新节点造成的第一个失衡祖先出发，利用“两步插入方向”判断失衡是直线形还是折线形，再用旋转在不改变 BST 中序顺序的前提下恢复平衡。

插入后若某个祖先的 BF 变为 2 或 -2 ，就把它作为失衡点处理。设该点为 A ，观察插入路径经过的前两条边：

方向	名称	修复方式
右 → 右	RR	对 A 左旋一次
左 → 右	LR	先对 A 的左孩子左旋，再对 A 右旋
右 → 左	RL	先对 A 的右孩子右旋，再对 A 左旋
左 → 左	LL	对 A 右旋一次

解释补充： 上表中 LL 可由课件给出的 RR 机制镜像得到；课件页面明确展开的是 RR、LR 和 RL 例子。

旋转只重组失衡点附近的节点和子树，不改变中序顺序，因此保持 BST 性质。 A 不一定是整棵树的根 [S5]。

详细机制

- RR：** 新节点位于失衡点的右孩子的右子树中。课件用 Mar, May, Nov 展示：Nov 使 Mar 的 BF 变为 -2 ，对 Mar 左旋后，May 成为局部根，Mar 和 Nov 分别成为其左右孩子，三个节点的 BF 恢复为 0 [S5]。
- LR：** 新节点先进入 A 的左孩子，再进入该孩子的右侧。课件用插入 Jan 后的月份树展

示：先左旋 A 的左孩子，使中间节点上升；再右旋 A ，让该中间节点成为局部根 [S7]。

3. **RL**：与 LR 镜像。新节点先进入 A 的右孩子，再进入该孩子的左侧；先右旋 A 的右孩子，再左旋 A [S8]。
4. **维护 BF**：插入可能使多个祖先的 BF 改变，即使不需要重构整棵树。课件也提示可改为在每个节点保存 height 字段 [S9]。

易错点

- BF 符号约定必须与公式一致，不要混用 $h_R - h_L$ 。
- 同向失衡用单旋，拐弯失衡必须双旋。
- 旋转后要检查局部的父子关系与子树归属，不能只调整两个键值。

来源：[S5-S9]

3. AVL 的高度界 [S10]

重要性：A · 必须掌握

核心原理

从“高度为 h 的 AVL 树至少需要多少节点”出发，用最小节点数的递推关系证明 AVL 高度是对数级。

设 n_h 是高度平衡树高度为 h 时的最小节点数。为使整棵树高度达到 h ，根的至少一个子树高度为 $h - 1$ ；另一个子树不能比它矮超过一层，因此至少为 $h - 2$ 。于是

$$n_h = n_{h-1} + n_{h-2} + 1.$$

课件给出 Fibonacci 数 (Fibonacci numbers)

$$F_0 = 0, \quad F_1 = 1, \quad F_i = F_{i-1} + F_{i-2} \quad (i > 1),$$

并给出

$$n_h = F_{h+2} - 1 \quad (h \geq 0).$$

补充推导：边界 $n_0 = 1, n_1 = 2$ 对应 $F_2 - 1 = 1, F_3 - 1 = 2$ ；由 Fibonacci 数的封闭形式

$$F_i \approx \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^i,$$

可得

$$n_h \approx \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^{h+2} - 1,$$

因此节点数关于高度指数增长，等价于

$$h = O(\ln n).$$

所以虽然 BST 操作是 $O(h)$ ，AVL 保证了 h 为对数级，从而搜索、插入和删除都有对数高度界 [S10]。

来源：[S10]

4. Splay Trees：访问后自调整 [S11-S17]

重要性：A · 必须掌握

4.1 目标与基本思想 [S11]

重要性：A · 必须掌握

Splay 树（Splay Tree）不承诺每次操作都 $O(\log N)$ ，而是要求：从空树开始的任意 M 个连续树操作总时间至多 $O(M \log N)$ 。因此单次访问仍可能花费 $O(N)$ ，但摊还时间为 $O(\log N)$ ，不存在坏的输入序列 [S11]。

基本思想是：节点被访问后，通过一系列旋转被推到根。这样最近访问的节点更接近根，路径上的许多节点深度也被压缩 [S11, S15]。

来源：[S11, S15]

4.2 为什么不能只把访问点逐层旋到根 [S12-S13]

重要性：B · 重点理解

课件先展示一种错误做法：访问 k_1 后只用单旋逐层上移。该过程不能有效压缩路径，图示结论为“Does NOT work!” [S12]。

更坏的情况是插入 $1, 2, \dots, N$ 后依次查找 $1, 2, \dots, N$ 。若每次只做简单上旋，总时间可达到

$$T(N) = O(N^2).$$

所以 Splay 必须使用下面成组的旋转模式，而不是简单地把访问节点逐层旋转 [S13]。

来源：[S12-S13]

4.3 Zig、Zig-zag 与 Zig-zig [S14-S16]

重要性：A · 必须掌握

设被访问节点为 X ，其父节点为 P ，祖父节点为 G [S14]。

情况	条件	操作
Zig	P 是根	旋转 X 与 P 一次
Zig-zag	$X - P$ 与 $P - G$ 方向相反	双旋
Zig-zig	$X - P$ 与 $P - G$ 方向相同	先处理 G, P, X 的单旋式重构

重复上述判断直到 X 到达根。课件强调：Splay 不只是把 X 移到根，还会大致把访问路径上大多数节点的深度减半 [S14-S15]。课件用插入 $1, 2, \dots, 7$ 后查找 1 的例子展示这一效果，并引用教材中的 32 节点例子作为扩展阅读 [S16]。

来源：[S14-S16]

4.4 Splay 删除 [S17]

重要性：A · 必须掌握

删除节点 X 的步骤如下：

1. 查找 X ，并通过 Splay 使 X 成为根；
2. 删除根 X ，得到左右子树 T_L, T_R ；
3. 对 T_L 执行 FindMax，使左子树最大元素成为 T_L 的根；
4. 把 T_R 接为该根的右孩子。

左子树最大元素没有右孩子，且大于 T_L 中所有元素、小于 T_R 中所有元素，因此连接后仍保持 BST 有序性 [S17]。

来源：[S17]

5. 摊还分析框架 [S18]

重要性：A · 必须掌握

摊还分析（Amortized Analysis）的目标是证明任意 M 个连续操作的总时间至多 $O(M \log N)$ 。课件给出三种方法：聚合分析（Aggregate Analysis）、记账法（Accounting Method）和势能法（Potential Method）[S18]。

三类界的强弱关系为

$$\text{worst-case bound} \geq \text{amortized bound} \geq \text{average-case bound}.$$

摊还分析不涉及概率，因此不能把摊还界理解成随机输入的平均时间 [S18]。

来源：[S18]

6. 三种摊还分析方法 [S19-S23]

6.1 聚合分析 [S19]

重要性：A · 必须掌握

聚合分析先证明 n 个操作在最坏情况下的总时间 $T(n)$ ，再得到每个操作的摊还成本 $T(n)/n$ [S19]。

核心原理

从“每个元素最多被压入一次、弹出一次”出发，说明看似昂贵的 MultiPop 序列总成本仍

由压入次数控制。

考虑从空栈开始的 n 个 `Push`、`Pop`、`MultiPop` 操作。`MultiPop(int k, Stack S)` 反复弹出直到栈空或已弹出 k 个元素，单次实际成本为

$$T = \min(\text{sizeof}(S), k).$$

虽然单次成本看似可达 $O(n)$ ，但每个对象每次被压入后最多被弹出一次，所以整段序列的总成本为 $O(n)$ ，摊还成本为

$$T_{\text{amortized}} = \frac{O(n)}{n} = O(1).$$

来源：[S19]

6.2 记账法 [S20-S21]

重要性：A · 必须掌握

核心原理

让高预付成本的操作把差额存成 credit，再由这些 credit 支付未来实际成本更高的操作，同时保证总预付不小于总实际成本。

设第 i 个操作的实际成本为 c_i ，摊还成本为 $\hat{c}_i$ 。对任意长度为 n 的操作序列，必须保证

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i, \quad T_{\text{amortized}} = \frac{\sum_{i=1}^n \hat{c}_i}{n}.$$

对栈操作设定：

$$c_i(\text{Push}) = 1, \quad c_i(\text{Pop}) = 1, \quad c_i(\text{MultiPop}) = \min(\text{sizeof}(S), k);$$

$$\hat{c}_i(\text{Push}) = 2, \quad \hat{c}_i(\text{Pop}) = 0, \quad \hat{c}_i(\text{MultiPop}) = 0.$$

每次 `Push` 支付实际成本 1 后，为栈中该对象留下 1 个 credit；`Pop` 或 `MultiPop` 弹出对象时消耗其 credit。因此从空栈开始，栈大小 $\text{sizeof}(S) \geq 0$ 保证总 credit 不为负，整段序列满足

$$O(n) = \sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i,$$

所以

$$T_{\text{amortized}} = O(n)/n = O(1).$$

注意不同操作的摊还成本可以不同 [S20-S21]。

来源: [S20-S21]

6.3 势能法 [S22-S23]

重要性: **A** · 必须掌握

核心原理

把记账法中的 credit 统一表示为数据结构状态的势能差, 使总摊还成本等于实际成本加初始、末态势能之差。

设 D_i 是第 i 个操作后的数据结构状态。势能函数 (Potential Function) Φ 满足

$$\hat{c}_i - c_i = \text{Credit}_i = \Phi(D_i) - \Phi(D_{i-1}),$$

于是

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0).$$

课件要求 $\Phi(D_n) - \Phi(D_0) \geq 0$, 并指出好的势能函数通常在序列开始时取得最小值 [S22]。

对栈取

D_i = 第 i 个操作后的栈, $\Phi(D_i)$ = 栈中对象数.

从空栈开始, $\Phi(D_i) \geq 0 = \Phi(D_0)$ 。若 `MultiPop` 实际弹出 k' 个元素, 则:

操作	势能变化	摊还成本
<code>Push</code>	+1	$1 + 1 = 2$

操作	势能变化	摊还成本
Pop	-1	$1 - 1 = 0$
MultiPop	$-k'$	$k' - k' = 0$

因此 $\sum \hat{c}_i = O(n)$, 且

$$T_{\text{amortized}} = O(n)/n = O(1).$$

来源: [S22-S23]

7. 用势能分析 Splay Trees [S24-S25]

重要性: A · 必须掌握

核心原理

用子树大小的对数定义节点的秩, 使访问前后的秩差既能抵消旋转的实际成本, 又能控制整个伸展过程的势能增幅。

对 Splay 树, 课件将 D_i 记为第 i 个操作后所得树的根, 并要求势能在 n 步内至多增加 $O(\log N)$, 同时抵消旋转次数: Zig 用 1 次旋转, Zig-zag 用 2 次, Zig-zig 用 2 次。为此定义

$$\Phi(T) = \sum_{i \in T} \log S(i),$$

其中 $S(i)$ 是以 i 为根的子树大小 (包含 i 本身)。令

$$R(i) = \log S(i),$$

则 $R(i)$ 称为节点/子树的秩 (Rank), 近似刻画树高 [S24]。课件提示不要直接使用树高: 旋转中秩差能形成可计算的上界, 而高度变化不易给出所需抵消关系。

解释补充: 直接用高度时, 一次旋转后多个节点高度的改变难以统一抵消旋转成本; 子树大小的对数在旋转中具有可加性, 便于把各节点秩差写成 $\hat{c}_i$ 的上界。

设 $R_1(v)$ 和 $R_2(v)$ 分别是某次旋转前后节点 v 的秩。三种旋转的摊还成本形式如下 [S25]:

旋转	摊还成本表达式	课件给出的界
Zig	$\hat{c}_i = 1 + R_2(X) - R_1(X) + R_2(P) - R_1(P)$	$\leq 1 + R_2(X) - R_1(X)$
Zig-zag	$\hat{c}_i = 2 + R_2(X) - R_1(X) + R_2(P) - R_1(P) + R_2(G) - R_1(G)$	$\leq 1 + 3(R_2(X) - R_1(X))$
Zig-zig	$\hat{c}_i = 2 + R_2(X) - R_1(X) + R_2(P) - R_1(P) + R_2(G) - R_1(G)$	$\leq 3(R_2(X) - R_1(X))$

课件引用 Lemma 11.4 [1] p.448 支撑 Zig-zag 的不等式，并给出定理：

$$\hat{c}_i \leq 3(R(T) - R(X)) + 1 = O(\log N),$$

其中 T 是伸展前树的根， X 是被伸展节点。沿途各旋转阶段的中间秩差相互抵消，最终只留下树根与被访问节点的秩差以及末尾 Zig 的常数 1 [S25]。

来源：[S24-S25]

8. 参考文献 [S26]

重要性：C · 了解即可

课件列出两本参考书：Weiss, *Data Structure and Algorithm Analysis in C*, 2nd Edition, Ch.4 p.106-128 与 Ch.11 p.447-451; Cormen et al., *Introduction to Algorithms*, 3rd Edition, Ch.17 p.451-478 [S26]。

来源：[S26]

本讲总结

- AVL 通过每个节点 $|h_L - h_R| \leq 1$ 的局部条件控制 BST 高度 [S4]。
- RR/LL 用单旋，LR/RL 用双旋；旋转保持 BST 中序顺序 [S5-S8]。
- 高度为 h 的 AVL 树至少有 $n_h = F_{h+2} - 1$ 个节点，因此 $h = O(\ln n)$ [S10]。
- Splay 树保证任意 M 个连续操作的摊还总界为 $O(M \log N)$ ，但单次操作可为 $O(N)$ [S11]。
- Zig、Zig-zag、Zig-zig 是正确伸展的基本旋转组合；简单逐层上旋会失败 [S13-S15]。

- 摊还分析不依赖概率，其界介于最坏情形界与平均情形界之间 [S18]。
- 栈的 `MultiPop` 可用聚合、记账、势能三种方法证明摊还成本 $O(1)$ [S19-S23]。
- Splay 的势能取 $\sum \log S(i)$ ，由此得到伸展摊还时间 $3(R(T) - R(X)) + 1 = O(\log N)$ [S24-S25]。

复杂度与适用条件速查

结构 / 方法	操作 / 结论	时间复杂度	条件	来源
BST	路径搜索	$O(h)$	h 可能退化为 $O(N)$	[S2]
AVL	插入后恢复平衡	$O(\log N)$ 高度界	每个节点 $BF \in \{-1, 0, 1\}$	[S4, S10]
Splay Tree	M 个连续操作	$O(M \log N)$ 总摊还	从空树开始；单次操作可 $O(N)$	[S11]
Stack + <code>MultiPop</code>	n 个操作总摊还	$O(1)$ / 操作	初始为空栈	[S19-S23]

复习自测

1. 为什么 AVL 只要求父子高度差不超过 1，就能得到对数高度？（来源：[S4, S10]）
2. LR 和 RL 为什么不能只做一次旋转？（来源：[S7-S8]）
3. Splay 树的单次操作可能 $O(N)$ ，为什么仍能说摊还时间为 $O(\log N)$ ？（来源：[S11]）
4. 记账法的核心不变量是什么？为什么 credit 不能为负？（来源：[S20-S21]）
5. Splay 势能函数为什么使用子树大小的对数而不是直接使用高度？（来源：[S24-S25]）

术语速查

术语	概念	来源
平衡因子 (Balance Factor,	$h_L - h_R$; AVL 中只能为 $-1, 0, 1$	[S4]

术语	概念	来源
BF)		
RR / LR / RL / LL 旋转	按插入路径两步方向选择的单旋或双旋	[S5-S8]
Splay 树 (Splay Tree)	访问后把节点伸展到根的自调整 BST	[S11]
Zig / Zig-zag / Zig-zig	父为根、方向相反、方向相同的三种伸展模式	[S14]
摊还分析 (Amortized Analysis)	证明操作序列总成本界，不涉及概率	[S18]
聚合分析 (Aggregate Analysis)	先证总成本，再除以操作数	[S19]
记账法 (Accounting Method)	用预付差额存 credit 支付后续成本	[S20]
势能法 (Potential Method)	用 $\Phi(D_i) - \Phi(D_{i-1})$ 表示 credit	[S22]
秩 (Rank)	$R(i) = \log S(i)$, $S(i)$ 为子树大小	[S24]

待核对

- 原始文件为二进制 .ppt ；本笔记使用 LibreOffice 转换出的 /tmp/ADS01AVL_Stu.pptx 及其渲染页图核对文本和公式。
- S12 的动画图示被提取为多层重叠图形，只保留了课件文字结论“Does NOT work!”和 S13 的 $O(N^2)$ 反例；未逐帧转写中间树形。
- S16 的具体 1, 2, ..., 7 树形动画与教材 32 节点扩展例子未逐节点复刻；如需逐步图示，请查看原课件 S16 或教材 Figures 4.52-4.60。